

Actividad Final Modulo DTPR

Data-Path Type R



Recamier Acevedo José de Jesús
García Torres Daniela Catalina
Espinoza Robles Fátima Gabriela
Lizaola Hernández Aylin Danae
Lomelí Mejía María del Sagrario
Ferrer Saucedo Valeria Lizeth
ARQUITECTURA DE COMPUTADORAS

1. Instrucciones Tipo R

Las instrucciones **Tipo R** son un formato de instrucción utilizado en el procesador MIPS. La letra "R" viene de *Register*, ya que todas las operaciones que realiza trabajan directamente sobre registros, sin acceder a memoria. Tienen un ancho de palabra fijo de 32 bits, y esos 32 bits se dividen en 6 campos:

- op (6 bits): el código de operación. Para instrucciones tipo R siempre es 000000.
- rs (5 bits): primer registro fuente.
- rt (5 bits): segundo registro fuente.
- rd (5 bits): registro destino donde se guarda el resultado.
- shamt (5 bits): cantidad de desplazamiento, usada solo en instrucciones de shift.
- funct (6 bits): indica la operación específica a realizar, ya que el opcode por sí solo no la define.

Las instrucciones que se manejan en este DataPath son

- ADD
- SUB
- AND
- OR
- SLT

Cada una tiene un código funct diferente que la ALUControl decodifica para decirle a la ALU que operación ejecutar.

La instrucción SLT (Set on Less Than)

SLT es una instrucción de comparación. Compara los valores almacenados en dos registros (rs y rt). Si el contenido de rs es menor que el de rt, se escribe 1 en el registro destino rd; de lo contrario, se escribe un 0.

Esta instrucción es especialmente útil para implementar estructuras de control, como condicionales y ciclos.

Su código funct es: 101010 (0x2A).

Ejemplo: `slt $t2, $t0, $t1`.

Si el valor en \$t0 es menor que el valor en \$t1, entonces: `$t2=1`.

En caso contrario: `$t2=0`.

Para implementar esta instrucción en Verilog, se utiliza el operador ternario, descrito en la siguiente sección.

2. Operacion Ternaria

La operación ternaria es una forma corta de escribir una condición if-else en una sola línea. En lugar de escribir varias líneas de código para decidir entre dos valores, se puede hacer todo en una expresión.

Su sintaxis es la siguiente:

```
condicion ? valor_si_verdadero : valor_si_falso
```

Si la condición se evalúa como verdadera (1), la expresión devuelve el primer valor. Si es falsa (0), devuelve el segundo. Es una operación que existe tanto en C/C++ como en Verilog.

Ejemplo en Verilog (SLT): result = (\$signed(A) < \$signed(B)) ? 32'd1 : 32'd0;

Esta línea revisa si A es menor que B (con signo). Si si, result toma el valor 1. Si no, toma el valor 0. Es exactamente lo que necesita la instrucción SLT.

3. Proceso de Compilación

La compilación es el proceso que convierte el código que escribe un programador (lenguaje de alto nivel como C) en instrucciones que el procesador puede entender y ejecutar. Este proceso no ocurre en un solo paso, sino que pasa por varias etapas:

Etapas del proceso

- **Preprocesamiento:** Se procesan las directivas del código como los #include o #define. Se prepara el código antes de compilar.
- **Análisis léxico:** El compilador lee el código y lo divide en tokens, que son las unidades mínimas del lenguaje como palabras clave, números, operadores, etc.
- **Análisis sintáctico:** Se verifica que los tokens estén bien ordenados según las reglas del lenguaje. Si hay un error de sintaxis, aquí se detecta.
- **Análisis semántico:** Se revisa que el código tenga sentido lógico, por ejemplo, que las variables estén declaradas antes de usarse o que los tipos de datos sean compatibles.
- **Generación de código intermedio:** El compilador genera una representación interna del programa, independiente del hardware. Facilita la optimización.
- **Optimización:** Se mejora el código intermedio para que el programa final sea más rápido o use menos memoria.
- **Generación de código máquina:** Por último, se produce el código binario que el procesador puede ejecutar directamente. En el caso de MIPS, esto serían las instrucciones de 32 bits que hemos estado usando.

Este proceso es importante para el contexto del DataPath porque las instrucciones que se cargan en la memoria de instrucciones son exactamente el resultado final de este proceso: código máquina en binario listo para ser ejecutado ciclo a ciclo por el procesador.

4. Resumen de VideoClase

1. Formato de instrucciones MIPS:

- **Tipo R** (aritméticas): operaciones como ADD, SUB, SLT. Campos: OP (6 bits), RS, RT, RD, shamt, function (6 bits). Ejemplo práctico con SUB usando un banco de registros y ALU.
- **Tipo I** (inmediatas, transferencia, branches): ej. ADDI. Incluye valor inmediato de 16 bits. Instrucciones como LW, SW, BEQ, BNE.
- **Tipo J** (saltos incondicionales): J. Usa 26 bits para la dirección de destino.

2. Arquitectura:

- Diferencia entre Harvard (memoria de datos e instrucciones separadas) y Von Neumann.
- Las etapas del pipeline: fetch, decode/register read, execute, memory, write back.
- Implementación comportamental en Verilog: decodificadores y multiplexores con case o switch.

5. Desarrollo y Metodología

Para la realización de esta práctica se siguió en una metodología estructurada de diseño, implementación, simulación y certificación del DataPath Tipo R en Verilog, tomando como base la arquitectura de MIPS de ciclo único.

Inicialmente se analizó el formato de las instrucciones de Tipo R para identificar los campos opcode, rs, rt, rd, shamt y funct, así como las señales necesarias para cada operación.

Posteriormente se diseñaron e implementaron los módulos principales del procesador:

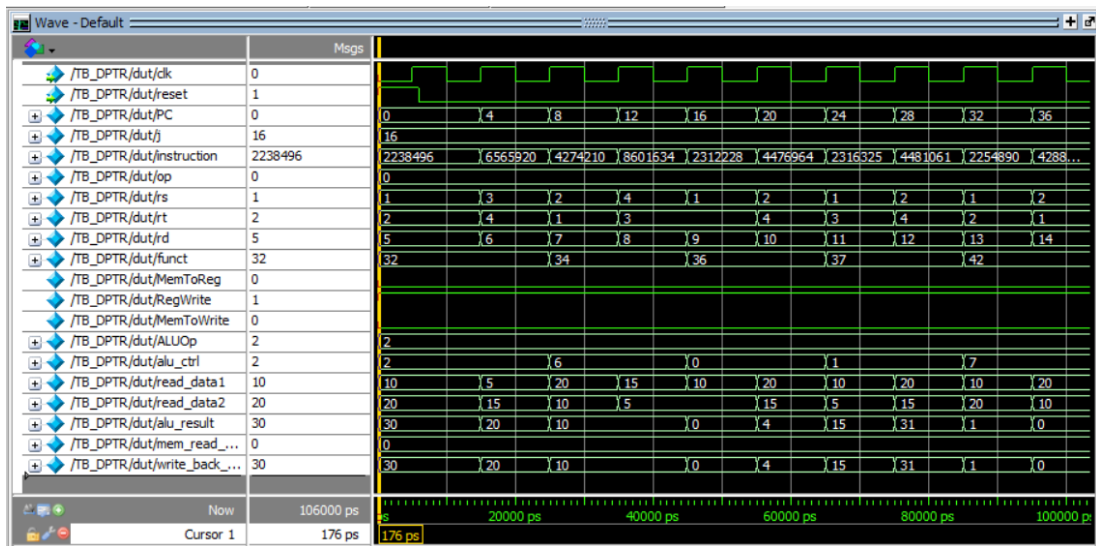
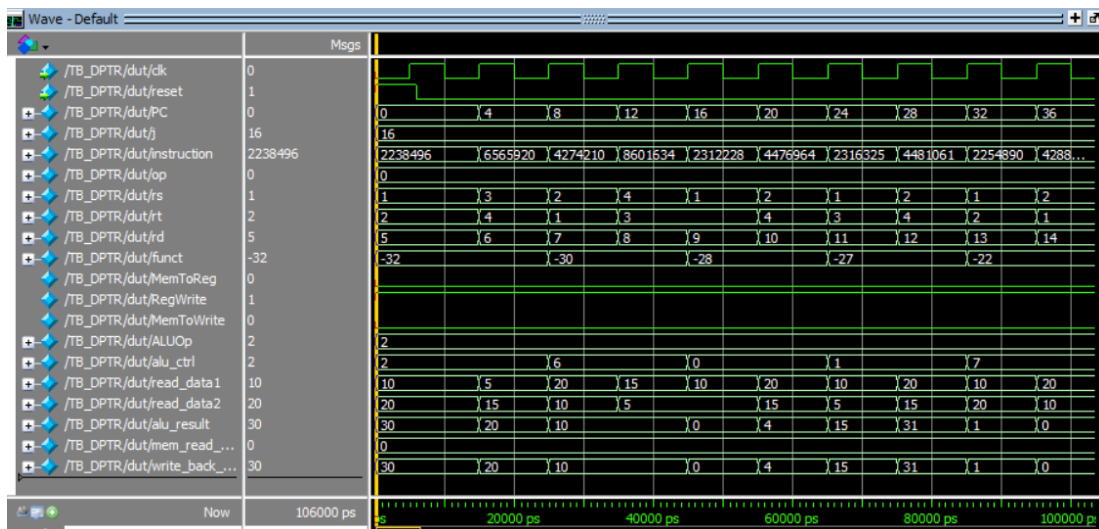
- Banco de registros

- ALU
- Unidad de control principal
- ALU Control
- Memoria de instrucciones
- Multiplexor

Cada módulo fue probado individualmente para verificar su funcionamiento antes de integrarlo al sistema completo.

La instrucción SLT se implemento dentro de la ALU utilizando el operador ternario y comparación con signo.

Tabla de instrucciones			
Instrucciones		Lenguaje maquina	
Instrucción	Ensamblador	Hexadecimal	decimal
add	add \$5, \$1, \$2	00222820	000000, 00001, 00010, 00101, 00000, 100000
add	add \$6, \$3, \$4	00643020	000000, 00011, 00100, 00110, 00000, 100000
sub	sub \$7, \$2, \$1	00413822	000000, 00010, 00001, 00111, 00000, 100010
sub	sub \$8, \$4, \$3	00834022	000000, 00100, 00011, 00100, 00000, 100010
and	and \$9, \$1, \$3	00234824	000000, 00001, 00011, 00101, 00000, 100100
and	and \$10, \$2, \$4	00445024	000000, 00010, 00100, 01010, 00000, 100100
or	or \$11, \$1, \$3	00235825	000000, 00001, 00011, 01011, 00000, 100101
or	or \$12, \$2, \$4	00446025	000000, 00001, 00011, 01011, 00000, 100101
slt	slt \$13, \$1 , \$2	0022682A	000000, 00001, 00010, 01101, 00000, 101010
slt	slt \$14, \$2, \$1	0041702A	000000, 00010, 00001, 0111, 000000, 101010



```
Transcript
#
# Tiempo(ns) | PC | Instruccion | ALU_Result (dec)
#
V$IM 4> run -all
#
# 15 ns | 0 | 0x00222820 | 30
# 25 ns | 4 | 0x00643020 | 20
# 35 ns | 8 | 0x00413822 | 10
# 45 ns | 12 | 0x00834022 | 10
# 55 ns | 16 | 0x00234824 | 0
# 65 ns | 20 | 0x00445024 | 4
# 75 ns | 24 | 0x00235825 | 15
# 85 ns | 28 | 0x00446025 | 31
# 95 ns | 32 | 0x0022682a | 1
# 105 ns | 36 | 0x0041702a | 0
#
# =====
# RESULTADOS DPTR - Formato Decimal
# =====
#
# Reg | Valor | Esperado | OK?
#
# $5 | 30 | 30 | OK
# $6 | 20 | 20 | OK
# $7 | 10 | 10 | OK
# $8 | 10 | 10 | OK
# $9 | 0 | 0 | OK
# $10 | 4 | 4 | OK
# $11 | 15 | 15 | OK
# $12 | 31 | 31 | OK
# $13 | 1 | 1 | OK
# $14 | 0 | 0 | OK
#
# =====
```

La implementación del DataPath Tipo R se realizó en Verilog HDL utilizando una arquitectura modular.

El módulo principal integra todos los componentes del procesador, permitiendo ejecutar instrucciones aritméticas y lógicas de tipo R.

Los módulos implementados fueron:

- **Banco de registros:** Almacena los operandos y resultados.
- **ALU:** Ejecuta las operaciones aritméticas y lógicas.
- **Unidad de Control:** Genera las señales de control principales.
- **ALU Control:** Determina la operación específica de la ALU a partir del campo funct.
- **Memoria de instrucciones:** Almacena el programa a ejecutar.
- **Multiplexores:** Seleccionan las rutas de datos adecuadas.

La ALU soporta las instrucciones ADD, SUB, AND, OR y SLT. Para SLT, se utilizó comparación con signo mediante el operador ternario.

Durante la simulación, se comprobó que cada instrucción actualizara correctamente el registro destino y produjera los resultados esperados.

Al realizar las pruebas las comprobamos con signo y sin signo, y para poder realizarlas

se pusieron en modo hexadecimal para mayor facilidad a la hora de probar diversos datos.

6. Conclusiones

Durante esta práctica se logró comprender el funcionamiento interno de las instrucciones aritméticas y lógicas, así como la interacción entre los distintos módulos que conforman un procesador.

Además, mediante la simulación en ModelSim, se verificó que las instrucciones se ejecutaran correctamente y nos dieran los resultados correctos.

Esta practica fortaleció significativamente la comprensión de la arquitectura de computadoras y del diseño de sistemas digitales mediante lenguajes de descripción de hardware.

7. Referencias

- Hennessy, J. L., & Patterson, D. A. (1993). Arquitectura de computadores: Un enfoque cuantitativo. McGraw-Hill Interamericana.
<https://elhacker.info/manuales/Lenguajes%20de%20Programacion/Arquitectura%20de%20computadores/ArquitecturadeComputadores-JohnL.Hennessy-DavidA.Patterson-UnEnfoqueCuantitativo-Capturado.pdf>
- Universidad Nacional Autónoma de México. (2023). Informática III: Material didáctico (LI_2325). Facultad de Contaduría y Administración, CEDIGEC.
https://cedigec.fca.unam.mx/materiales/plan_2023/informatica/3/LI_2325.pdf
- Trini, S. (2020). MIPS: Datapath. la35.net. <https://la35.net/orga/mips-datapath.html>
- MIPS Technologies. (2016). *MIPS32 architecture for programmers: Volume II: The MIPS32 instruction set*. MIPS Technologies. <https://s3-eu-west-1.amazonaws.com/downloads-mips/documents/MD00086-2B-MIPS32BIS-AFP-6.06.pdf>
- freeCodeCamp. (2024). Operador ternario en C: cómo usar el operador condicional. freeCodeCamp. <https://www.freecodecamp.org/espanol/news/operador-ternario-en-c/>